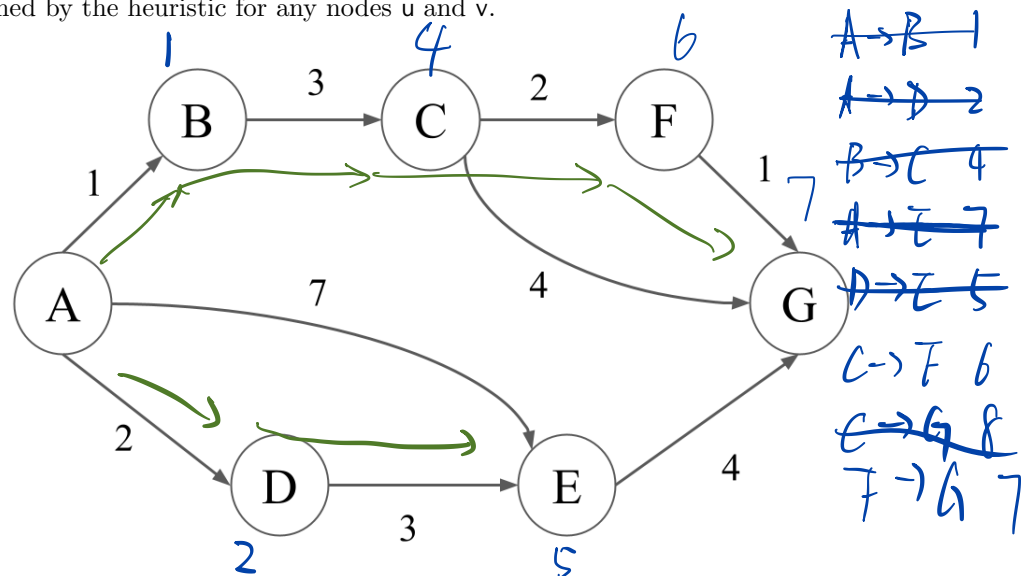


1 The Shortest Path To Your Heart

For the graph below, let $g(u, v)$ be the weight of the edge between any nodes u and v . Let $h(u, v)$ be the value returned by the heuristic for any nodes u and v .



Below, the pseudocode for Dijkstra's and A* are both shown for your reference throughout the problem.

Dijkstra's Pseudocode

```

1 PQ = new PriorityQueue()
2 PQ.add(A, 0)
3 PQ.add(v, infinity) # (all nodes except A).
4
5 distTo = {} # map
6 distTo[A] = 0
7 distTo[v] = infinity # (all nodes except A).
8
9 while (not PQ.isEmpty()):
10     popNode, popPriority = PQ.pop()
11
12     for child in popNode.children:
13         if PQ.contains(child):
14             potentialDist = distTo[popNode] +
15                 edgeWeight(popNode, child)
16             if potentialDist < distTo[child]:
17                 distTo.put(child, potentialDist)
18                 PQ.changePriority(child, potentialDist)

```

A* Pseudocode

```

1 PQ = new PriorityQueue()
2 PQ.add(A, h(A))
3 PQ.add(v, infinity) # (all nodes except A).
4
5 distTo = {} # map
6 distTo[A] = 0
7 distTo[v] = infinity # (all nodes except A).
8
9 while (not PQ.isEmpty()):
10     poppedNode, poppedPriority = PQ.pop()
11     if (poppedNode == goal): terminate
12
13     for child in poppedNode.children:
14         if PQ.contains(child):
15             potentialDist = distTo[poppedNode] +
16                 edgeWeight(poppedNode, child)
17
18             if potentialDist < distTo[child]:
19                 distTo.put(child, potentialDist)
20                 PQ.changePriority(child, potentialDist + h(child))

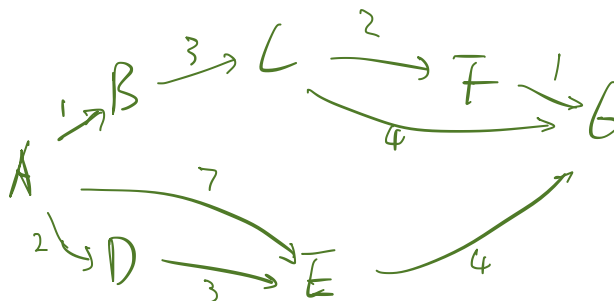
```

- (a) Run Dijkstra's algorithm to find the shortest paths from A to every other vertex. You may find it helpful to keep track of the priority queue. We have provided a table to keep track of best distances, and the edge leading to each vertex on the currently known shortest paths.

	A	B	C	D	E	F	G
DistTo	0	1	4	2	5	6	7
EdgeTo	X	A	B	A	D	C	F

- (b) *Extra:* Given the weights and heuristic values for the graph above, what path would A^* search return, starting from A and with G as a goal? Note that the edge weights provided below for your convenience are the same as in the image.

Edge weights	Heuristics
$g(A, B) = 1$	$h(A, G) = 7$
$g(B, C) = 3$	$h(B, G) = 6$
$g(C, F) = 2$	$h(C, G) = 3$
$g(C, G) = 4$	$h(F, G) = 1$
$g(F, G) = 1$	$h(D, G) = 6$
$g(A, D) = 2$	$h(E, G) = 3$
$g(D, E) = 3$	
$g(E, G) = 4$	
$g(A, E) = 7$	

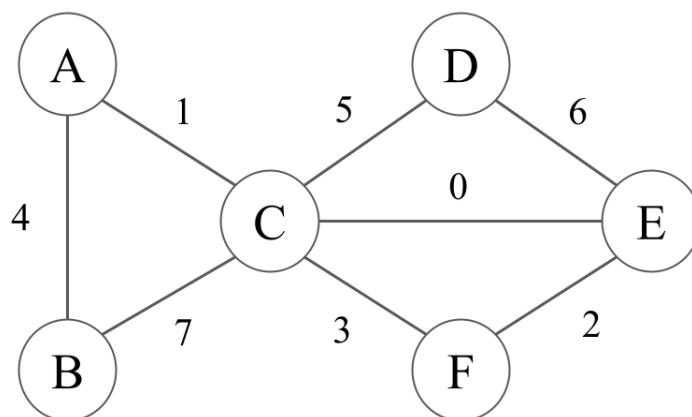


	A	B	C	D	E	F	G
DistTo	0	1	4			6	7
EdgeTo	X	A	B			C	F

~~$A \rightarrow B$ 1+6~~
 $A \rightarrow D$ 2+6
 $A \rightarrow E$ 7+3
 ~~$B \rightarrow C$ 4+3~~
 ~~$E \rightarrow F$ 6+1~~
 ~~$C \rightarrow G$ 8+0~~
 $F \rightarrow G$ 7+0

2 Minimalist Moles

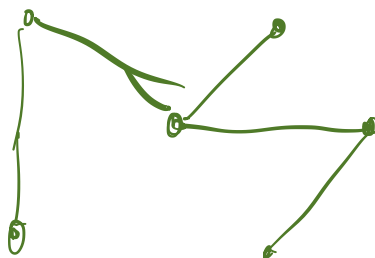
Mindy the mole wants to dig a network of tunnels connecting all of their secret hideouts. There are a set few paths between the secret hideouts that Mindy can choose to possibly include in their tunnel system, shown below. However, some portions of the ground are harder to dig than others, and Mindy wants to do as little work as possible. In the diagram below, the numbers next to the paths correspond to how hard that path is to dig for Mindy.



- (a) How can Mindy figure out a tunnel system to connect their secret hideouts while doing minimal work?

MST minimal spanning Tree

- (b) *Extra:* Find a valid MST for the graph above using Kruskal's algorithm, then Prim's. For Prim's algorithm, take A as the start node. In both cases, if there is ever a tie, choose the edge that connects two nodes with lower alphabetical order.



- (c) *Extra:* Are the above MSTs different or the same? Is there a different tie-breaking scheme that would change your answer?

Only happen when duplicate weight of edges.